Пензенский государственный университет
Факультет вычислительной техники
Кафедра «Вычислительная техника»

Многопоточность в Java

Отчет о лабораторной работе № 5
по дисциплине «Разработка кроссплатформенных приложений»
Вариант 1

Выполнили: студентыгр.23ВОЭ1
Чиндин И.Д.
Снигирев А.А.
Проверили: к.т.н., доцент
Юрова О.В.
к.т.н., доцент
Карамышева Н.С.

2026

1 Цель работы

Научиться создавать многопоточные приложения с использованием стандартных средств языка Java.

2 Формулировка задачи

Модифицировать приложение из предыдущей лабораторной работы, реализовав вычисление определенного интеграла в нескольких дополнительных потоках (число потоков определяется номером варианта), снимая нагрузку с основного потока и предотвращая "подвисание" графического интерфейса. Реализовать многопоточность путем наследования от класса Thread.

3 Ход работы

Для реализации потока, который будет рассчитывать функцию RecIntegral, необходимо создать новый класс, который наследуется он класса Thread.

Класс NewThread наследуется от Thread и используется для выполнения вычисления определённого интеграла в отдельном потоке, чтобы не нагружать основной графический интерфейс. При создании объекта этого класса в конструктор передаются три параметра: объект RecIntegral, содержащий данные для вычисления; индекс строки таблицы, которую нужно обновить; и модель таблицы DefaultTableModel, через которую будет записан результат.

Когда поток запускается методом start(), автоматически вызывается его метод run(). В начале работы run() отображается окно с сообщением «Поток открылся», что подтверждает успешный запуск фонового вычисления. Затем поток вызывает метод CalcIntegral() у объекта RecIntegral, передавая ему нижний предел, верхний предел и шаг. Полученное значение интеграла сохраняется в объекте с помощью setResult().

Так как обновлять элементы графического интерфейса напрямую из фонового потока нельзя, используется вызов SwingUtilities.invokeLater(). Этот вызов передаёт задачу обновления таблицы обратно в основной поток Swing. Внутри него вызывается метод setValueAt(), который записывает вычисленный результат в соответствующую ячейку таблицы. Таким образом, интерфейс остаётся отзывчивым, а вычисления выполняются в отдельном потоке без зависаний.

Класс InvalidRec представляет собой пользовательское исключение, предназначенное для обработки недопустимых параметров интегрирования в программе. Он объявлен как наследник базового класса Exception.

Конструктор класса InvalidRec принимает строковый параметр message и передаёт его в конструктор класса Exception с помощью вызова super(message). При возникновении ошибки можно перехватить это исключение в блоке try-catch и вывести пользователю сообщение. Листинг приведен в приложении А.

Так же необходимо изменить конструктор класса RecIntegral. В обоих конструкторах класса выполняется проверка всех переданных значений с помощью метода isValid(). Если хотя бы один параметр выходит за допустимые границы от 0.000001 до 1 000 000 генерируется пользовательское исключение InvalidRec с сообщением о ошибке. Первый конструктор инициализирует пределы и шаг, устанавливая результат равным нулю, а второй позволяет задать и уже вычисленное значение результата.

Метод isValid()возвращает логическое значение, указывающее, находится ли число в допустимом диапазоне.

Так же необходимо изменить методы обработчиков событий и добавить к кнопке «Добавить» обработчик исключений. Обработчик кнопки «Добавить» обеспечивает ввод новых данных в таблицу. При нажатии выполняется проверка: если хотя бы одно из текстовых полей пустое, пользователю выводится сообщение «Заполните все поля!». В случае ошибки формата отображается сообщение «Введите корректные числовые значения!». Если

данные корректны, создаётся объект RecIntegral, который проходит проверку допустимости значений в своём конструкторе. Затем новая строка с параметрами интегрирования и нулевым результатом добавляется в таблицу, а объект сохраняется в коллекции arrInteg. После добавления поля ввода очищаются.

Обработчик кнопки «Рассчитать» начинает свою работу с получения модели таблицы DefaultTableModel и определения выбранной пользователем строки. Если строка не выбрана, пользователю выводится сообщение «Выберите строку!» и дальнейшая работа метода прекращается. После этого из выбранной строки таблицы извлекаются три значения: нижний предел, верхний предел и шаг. Эти данные преобразуются в числа типа double. На основе полученных значений создаётся новый объект RecIntegral, который представляет собой набор параметров для вычисления определённого интеграла. Созданный объект сохраняется в коллекции arrInteg на место соответствующей строки. Вместо того чтобы выполнять вычисление интеграла прямо в обработчике, как это было раньше, программа создаёт новый объект класса NewThread, передавая ему объект RecIntegral, индекс строки и модель таблицы. Затем вызывается метод start(), который запускает вычисление в отдельном потоке. Если при вводе данных возникает ошибка формата или значения выходят за допустимые пределы, соответствующее исключение перехватывается, и пользователю выводится сообщение об ошибке. Листинг приведён в приложении А.

Обработчик кнопки «Удалить» позволяет удалить выбранную строку таблицы. Он получает индекс выделенной строки, и если строка не выбрана, выводит предупреждение «Enterstring». Если строка выделена, она удаляется из таблицы методом removeRow(rowNum), а соответствующий объект удаляется из коллекции arrInteg.

Обработчик кнопки «Очистить» полностью очищает таблицу, устанавливая количество строк равным нулю. При этом коллекция arrInteg не

очищается, что позволяет при необходимости восстановить данные обратно в таблицу. Листинг приведён в приложении А.

Обработчик кнопки «Заполнить» выполняет обратную операцию — он восстанавливает содержимое таблицы из коллекции arrInteg. При нажатии таблица предварительно очищается, после чего для каждого объекта интеграла добавляется строка, содержащая его параметры и ранее вычисленный результат. Это позволяет пользователю повторно загрузить данные после очистки таблицы.

Обработчик кнопки «Сохр.текст» начинает свою работу с создания нового объекта JFileChooser, который открывает диалоговое окно выбора файла. Затем вызывается метод showSaveDialog(this), который отображает окно сохранения; если пользователь нажимает кнопку «Сохранить», то есть результат равен APPROVE_OPTION, программа переходит к выполнению блока try. Внутри блока создаётся объект FileWriter, связанный с выбранным пользователем файлом, и он автоматически закрывается после завершения работы. Далее запускается цикл for, который проходит по каждому объекту RecIntegral в коллекции arrInteg. Для каждого элемента вызываются методы getLowLim(), getUpLim(), getStep() и geResult(), и их значения записываются в файл в виде строки, где значения разделены пробелами, а каждая запись заканчивается символом новой строки. После успешной записи всех данных пользователю показывается диалоговое окно с сообщением «Файл успешно сохранён!». Если в процессе сохранения возникает ошибка, она перехватывается блоком catch, и пользователю выводится сообщение об ошибке.

Обработчик кнопки «Сохр. двоичный» начинает работу с создания JFileChooser и отображения окна сохранения файла через showSaveDialog(this). Если пользователь подтверждает выбор, программа входит в блок try, где создаётся объект ObjectOutputStream. Благодаря конструкции try-catch поток автоматически закрывается после завершения записи. Затем вызывается метод writeObject(arrInteg), который сериализует всю коллекцию arrInteg и записывает

её в бинарном виде в файл. После успешного завершения операции пользователю показывается сообщение «Данные сохранены (двоичный формат)». Если при записи возникает исключение, оно перехватывается блоком catch, и пользователю выводится сообщение об ошибке с текстом возникшего исключения. Листинг приведён в приложении А.

Обработчик кнопки «Загр. текст» начинает работу с создания JFileChooser и вызова showOpenDialog(this), который открывает окно выбора файла. Если пользователь выбирает файл и подтверждает действие, программа сразу выводит диалоговое окно JOptionPane.showConfirmDialog, предлагая очистить текущую коллекцию. Если пользователь выбирает «Да», коллекция arrInteg очищается методом clear(), а таблица очищается. После этого программа входит в блок try, где создаётся BufferedReader, читающий выбранный файл построчно. Далее получаем модель таблицы DefaultTableModel, чтобы добавлять в неё строки. Затем запускается цикл, который читает файл строка за строкой; каждая строка разбивается методом split(" ") на четыре части: нижний предел, верхний предел, шаг и результат. Эти значения преобразуются в числа типа double, после чего создаётся новый объект RecIntegral, который добавляется в коллекцию arrInteg. Одновременно в таблицу добавляется новая строка с этими же значениями. После завершения чтения файла пользователю показывается сообщение «Файл успешно загружен!». Если в процессе чтения возникает ошибка, она перехватывается блоком catch, и пользователю выводится сообщение об ошибке. Листинг приведен в приложении А.

Обработчик кнопки «Загр. двоичный» работает аналогично загрузке текстового файла, но использует бинарную десериализацию. Сначала создаётся JFileChooser, и если пользователь выбирает файл, программа выводит диалоговое окно с вопросом об очистке текущей коллекции. Если пользователь выбирает «Да», коллекция arrInteg очищается, а таблица сбрасывается. Затем программа входит в блок try, где создаётся ObjectInputStream, читающий бинарный файл. Из потока считывается объект, который приводится к типу

ArrayList<RecIntegral> и сохраняется во временную переменную loaded. Далее получаем модель таблицы DefaultTableModel, после чего запускается цикл, который проходит по всем элементам загруженной коллекции. Каждый объект добавляется в arrInteg, а его данные — в таблицу в виде новой строки. После успешной загрузки пользователю показывается сообщение «Данные загружены (двоичный формат)». Если возникает ошибка, она перехватывается блоком catch, и пользователю выводится сообщение об ошибке. Листинг приведен в приложении А.

Результаты выполнения программы приведены в приложении Б (рисунок Б.1).

Вывод

В ходе выполения лабораторной работы модифицировано приложение из предыдущей лабораторной работы, реализовано вычисление определенного интеграла в дополнительном потоке, снимая нагрузку с основного потока и предотвращая "подвисание" графического интерфейса. Реализована многопоточность путем наследования от класса Thread.

ТЕКСТ ПРОГРАММЫ

Приложение А

(обязательное)

Основная программа:

```
package lab1.pkg1;

import  javax.swing.table.DefaultTableModel;
import  javax.swing.JOptionPane;
import java.util.ArrayList;
import javax.swing.JFileChooser;
import java.io.File;
import java.io.FileWriter;
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.ObjectOutputStream;
import java.io.ObjectInputStream;
import java.io.FileOutputStream;
import java.io.FileInputStream;


/**
 *
 * @author student
 */
public class NewJFrame extends javax.swing.JFrame {
ArrayList<RecIntegral> arrInteg = new ArrayList<>();
   /**
    * Creates new form NewJFrame
    */
   public NewJFrame() {
      initComponents();
   }

   /**
```

```
 * This method is called from within the constructor to initialize the form.
 * WARNING: Do NOT modify this code. The content of this method is always
 * regenerated by the Form Editor.
 */
@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    jFrame1 = new javax.swing.JFrame();
    jPanel1 = new javax.swing.JPanel();
    jScrollPane1 = new javax.swing.JScrollPane();
    jTable1 = new javax.swing.JTable();
    jButtonAdd = new javax.swing.JButton();
    jButtonDelete = new javax.swing.JButton();
    jButtonResult = new javax.swing.JButton();
    jTextFieldLow = new javax.swing.JTextField();
    jTextFieldUp = new javax.swing.JTextField();
    jTextFieldStep = new javax.swing.JTextField();
    jLabel1 = new javax.swing.JLabel();
    jLabel2 = new javax.swing.JLabel();
    jLabel3 = new javax.swing.JLabel();
    jLabel4 = new javax.swing.JLabel();
    AddColection = new javax.swing.JButton();
    DeleteAll = new javax.swing.JButton();
    SaveText = new javax.swing.JButton();
    SaveBinary = new javax.swing.JButton();
    DownloadBinary = new javax.swing.JButton();
    DownloadText = new javax.swing.JButton();

    javax.swing.GroupLayout jFrame1Layout = new javax.swing.GroupLayout(jFrame1.getContentPane());
    jFrame1.getContentPane().setLayout(jFrame1Layout);
```

```
jFrame1Layout.setHorizontalGroup(
    jFrame1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGap(0, 400, Short.MAX_VALUE)
);
jFrame1Layout.setVerticalGroup(
    jFrame1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGap(0, 300, Short.MAX_VALUE)
);

setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
setBackground(new java.awt.Color(255, 153, 0));
setBounds(new java.awt.Rectangle(0, 0, 10, 10));
setCursor(new java.awt.Cursor(java.awt.Cursor.DEFAULT_CURSOR));
setForeground(new java.awt.Color(255, 255, 255));
addComponentListener(new java.awt.event.ComponentAdapter() {
    public void componentHidden(java.awt.event.ComponentEvent evt) {
        formComponentHidden(evt);
    }
});

jPanel1.setBackground(new java.awt.Color(204, 255, 204));
jPanel1.setForeground(new java.awt.Color(204, 0, 51));

jScrollPane1.setForeground(new java.awt.Color(255, 0, 102));

jTable1.setBackground(new java.awt.Color(204, 204, 204));
jTable1.setBorder(new javax.swing.border.MatteBorder(null));
jTable1.setModel(new javax.swing.table.DefaultTableModel(
    new Object [][] {

    },
```

```
    new String [] {
        "Нижний порог", "Верхний порог", "Шаг", "Результат"
    }
) {
    boolean[] canEdit = new boolean [] {
        true, true, true, false
    };

    public boolean isCellEditable(int rowIndex, int columnIndex) {
        return canEdit [columnIndex];
    }
});
jScrollPane1.setViewportView(jTable1);

jButtonAdd.setBackground(new java.awt.Color(204, 204, 204));
jButtonAdd.setText("Добавить");
jButtonAdd.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButtonAddActionPerformed(evt);
    }
});

jButtonDelete.setBackground(new java.awt.Color(204, 204, 204));
jButtonDelete.setText("Удалить");
jButtonDelete.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButtonDeleteActionPerformed(evt);
    }
});

jButtonResult.setBackground(new java.awt.Color(204, 204, 204));
```

```
jButtonResult.setText("Рассчитать");
jButtonResult.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButtonResultActionPerformed(evt);
    }
});

jLabel1.setText("Нижний порог");

jLabel2.setText("Верхний порог");

jLabel3.setText("Шаг");

jLabel4.setText("1/x");

AddColection.setBackground(new java.awt.Color(204, 204, 204));
AddColection.setText("Заполнить");
AddColection.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        AddColectionActionPerformed(evt);
    }
});

DeleteAll.setBackground(new java.awt.Color(204, 204, 204));
DeleteAll.setText("Очистить");
DeleteAll.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        DeleteAllActionPerformed(evt);
    }
});
```

```
SaveText.setBackground(new java.awt.Color(204, 204, 204));
SaveText.setText("Сохр. текст");
SaveText.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        SaveTextActionPerformed(evt);
    }
});

SaveBinary.setBackground(new java.awt.Color(204, 204, 204));
SaveBinary.setText("Сохр. двоичный");
SaveBinary.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        SaveBinaryActionPerformed(evt);
    }
});

DownloadBinary.setBackground(new java.awt.Color(204, 204, 204));
DownloadBinary.setText("Загр. двоичный");
DownloadBinary.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        DownloadBinaryActionPerformed(evt);
    }
});

DownloadText.setBackground(new java.awt.Color(204, 204, 204));
DownloadText.setText("Загр. текст");
DownloadText.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        DownloadTextActionPerformed(evt);
    }
});
```

```
javax.swing.GroupLayout jPanel1Layout = new javax.swing.GroupLayout(jPanel1);
jPanel1.setLayout(jPanel1Layout);
jPanel1Layout.setHorizontalGroup(
    jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addComponent(jScrollPane1)
    .addGroup(jPanel1Layout.createSequentialGroup()
        .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
            .addGroup(javax.swing.GroupLayout.Alignment.LEADING,
jPanel1Layout.createSequentialGroup()
                .addGap(21, 21, 21)
                .addComponent(jLabel4))
            .addGroup(jPanel1Layout.createSequentialGroup()
                .addGap(243, 243, 243)
                .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)
                    .addComponent(DownloadBinary, javax.swing.GroupLayout.PREFERRED_SIZE, 117,
javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addComponent(SaveBinary, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                    .addComponent(DownloadText, javax.swing.GroupLayout.PREFERRED_SIZE, 117,
javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addComponent(SaveText, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))))
        .addGap(36, 36, 36)
        .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING, false)
            .addComponent(jButtonResult, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
            .addComponent(AddColection, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
            .addComponent(jButtonDelete, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
            .addComponent(jButtonAdd, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
```

```
                .addComponent(DeleteAll, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
            .addGap(0, 50, Short.MAX_VALUE))
        .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(jPanel1Layout.createSequentialGroup()
                .addContainerGap(78, Short.MAX_VALUE)
                .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                    .addComponent(jLabel1)
                    .addComponent(jLabel3)
                    .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
                        .addComponent(jTextFieldUp, javax.swing.GroupLayout.PREFERRED_SIZE, 133,
javax.swing.GroupLayout.PREFERRED_SIZE)
                        .addComponent(jTextFieldStep, javax.swing.GroupLayout.PREFERRED_SIZE, 133,
javax.swing.GroupLayout.PREFERRED_SIZE)
                        .addComponent(jTextFieldLow, javax.swing.GroupLayout.PREFERRED_SIZE, 133,
javax.swing.GroupLayout.PREFERRED_SIZE))
                    .addComponent(jLabel2))
                .addGap(328, 328, 328)))
    );
    jPanel1Layout.setVerticalGroup(
        jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, jPanel1Layout.createSequentialGroup()
            .addContainerGap()
            .addComponent(jLabel4)
            .addGap(23, 23, 23)
            .addComponent(jButtonAdd)
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
            .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                .addComponent(SaveText)
                .addComponent(jButtonDelete))
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 8, Short.MAX_VALUE)
            .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                .addComponent(SaveBinary)
```

```
                .addComponent(jButtonResult))
            .addGap(48, 48, 48)
            .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                .addComponent(DownloadText)
                .addComponent(AddColection))
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
            .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                .addComponent(DeleteAll)
                .addComponent(DownloadBinary))
            .addGap(65, 65, 65)
            .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 222,
javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(jPanel1Layout.createSequentialGroup()
                .addGap(68, 68, 68)
                .addComponent(jLabel1)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                .addComponent(jTextFieldLow, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
                .addComponent(jLabel2)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                .addComponent(jTextFieldUp, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
                .addComponent(jLabel3)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                .addComponent(jTextFieldStep, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                .addContainerGap(284, Short.MAX_VALUE)))
    );


    javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
```

```
        getContentPane().setLayout(layout);
        layout.setHorizontalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(jPanel1, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        );
        layout.setVerticalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(layout.createSequentialGroup()
                .addComponent(jPanel1, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                .addGap(0, 5, Short.MAX_VALUE))
        );

        pack();
    }// </editor-fold>

    private void jButtonAddActionPerformed(java.awt.event.ActionEvent evt) {
if (jTextFieldLow.getText().isEmpty() ||
        jTextFieldUp.getText().isEmpty() ||
        jTextFieldStep.getText().isEmpty())
    {
        JOptionPane.showMessageDialog(this, "Заполните все поля!");
        return;
    }

    try {
        double lowLim = Double.parseDouble(jTextFieldLow.getText());
        double upLim = Double.parseDouble(jTextFieldUp.getText());
        double step = Double.parseDouble(jTextFieldStep.getText());

        RecIntegral rec = new RecIntegral(lowLim, upLim, step);
```

```
    DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();
    tModel.addRow(new Object[]{lowLim, upLim, step, 0.0});

    arrInteg.add(rec);

    jTextFieldLow.setText("");
    jTextFieldUp.setText("");
    jTextFieldStep.setText("");

} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(this, "Введите корректные числовые значения!"  );
} catch (InvalidRec ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage());
}
}

private void jButtonResultActionPerformed(java.awt.event.ActionEvent evt) {
        try {
    DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();
    int rowNum = jTable1.getSelectedRow();

    if (rowNum == -1) {
        JOptionPane.showMessageDialog(this, "Выберите строку!");
        return;
    }

    double lowLim = Double.parseDouble(tModel.getValueAt(rowNum, 0).toString());
    double upLim = Double.parseDouble(tModel.getValueAt(rowNum, 1).toString());
    double step = Double.parseDouble(tModel.getValueAt(rowNum, 2).toString());
```

```
        RecIntegral rec = new RecIntegral(lowLim, upLim, step);

        arrInteg.set(rowNum, rec);
        NewThread thread = new NewThread(rec, rowNum, tModel);
        thread.start();

    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(this, "Некорректные данные в таблице!");
    } catch (InvalidRec ex) {
        JOptionPane.showMessageDialog(this, ex.getMessage());
    }


// TODO add your handling code here:
    }

    private void formComponentHidden(java.awt.event.ComponentEvent evt) {
        // TODO add your handling code here:
    }

    private void jButtonDeleteActionPerformed(java.awt.event.ActionEvent evt) {

    DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();
    int rowNum = jTable1.getSelectedRow();

    if (rowNum == -1) {
        JOptionPane.showMessageDialog(null, "Выберите строку!");
        return;
    }

    double low = Double.parseDouble(tModel.getValueAt(rowNum, 0).toString());
```

```
double up = Double.parseDouble(tModel.getValueAt(rowNum, 1).toString());
double step = Double.parseDouble(tModel.getValueAt(rowNum, 2).toString());

tModel.removeRow(rowNum);

arrInteg.removeIf(rec ->
        rec.getLowLim() == low &&
        rec.getUpLim() == up &&
        rec.getStep() == step
);
}

private void DeleteAllActionPerformed(java.awt.event.ActionEvent evt) {
   DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();
tModel.setRowCount(0);
}

private void AddColectionActionPerformed(java.awt.event.ActionEvent evt) {
   DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();
tModel.setRowCount(0);

for (RecIntegral rec : arrInteg) {
    tModel.addRow(new Object[]{
        rec.getLowLim(),
        rec.getUpLim(),
        rec.getStep(),
        rec.geResult()
    });
}
}
```

```
private void SaveTextActionPerformed(java.awt.event.ActionEvent evt) {

JFileChooser chooser = new JFileChooser();
if (chooser.showSaveDialog(this) == JFileChooser.APPROVE_OPTION) {
   try (FileWriter fw = new FileWriter(chooser.getSelectedFile())) {

      for (RecIntegral rec : arrInteg) {
         fw.write(rec.getLowLim() + " "
               + rec.getUpLim() + " "
               + rec.getStep() + " "
               + rec.geResult() + "\n");
      }

      JOptionPane.showMessageDialog(this, "Файл успешно сохранён!");
   } catch (Exception ex) {
      JOptionPane.showMessageDialog(this, "Ошибка сохранения: " + ex.getMessage());
   }
}
}

private void SaveBinaryActionPerformed(java.awt.event.ActionEvent evt) {
  JFileChooser chooser = new JFileChooser();
if (chooser.showSaveDialog(this) == JFileChooser.APPROVE_OPTION) {
   try (ObjectOutputStream out = new ObjectOutputStream(
         new FileOutputStream(chooser.getSelectedFile()))) {

      out.writeObject(arrInteg);
      JOptionPane.showMessageDialog(this, "Данные сохранены (двоичный формат)");

   } catch (Exception ex) {
      JOptionPane.showMessageDialog(this, "Ошибка сохранения: " + ex.getMessage());
```

```
    }
}
}

private void DownloadTextActionPerformed(java.awt.event.ActionEvent evt) {
    JFileChooser chooser = new JFileChooser();
if (chooser.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {

    int choice = JOptionPane.showConfirmDialog(
            this,
            "Очистить исходную коллекцию?",
            "Подтверждение",
            JOptionPane.YES_NO_OPTION
    );

    if (choice == JOptionPane.YES_OPTION) {
        arrInteg.clear();
        ((DefaultTableModel) jTable1.getModel()).setRowCount(0);
    }

    try (BufferedReader br = new BufferedReader(new FileReader(chooser.getSelectedFile()))) {

        DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();

        String line;
        while ((line = br.readLine()) != null) {
            String[] p = line.split(" ");
            double low = Double.parseDouble(p[0]);
            double up = Double.parseDouble(p[1]);
            double step = Double.parseDouble(p[2]);
            double res = Double.parseDouble(p[3]);
```

```
            RecIntegral rec = new RecIntegral(low, up, step, res);
            arrInteg.add(rec);

            tModel.addRow(new Object[]{low, up, step, res});
        }

        JOptionPane.showMessageDialog(this, "Файл успешно загружен!");
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Ошибка загрузки: " + ex.getMessage());
    }
}
}

private void DownloadBinaryActionPerformed(java.awt.event.ActionEvent evt) {
 JFileChooser chooser = new JFileChooser();
if (chooser.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {

    int choice = JOptionPane.showConfirmDialog(
            this,
            "Очистить исходную коллекцию?",
            "Подтверждение",
            JOptionPane.YES_NO_OPTION
    );

    if (choice == JOptionPane.YES_OPTION) {
        arrInteg.clear();
        ((DefaultTableModel) jTable1.getModel()).setRowCount(0);
    }

    try (ObjectInputStream in = new ObjectInputStream(
```

```
            new FileInputStream(chooser.getSelectedFile()))) {

        ArrayList<RecIntegral> loaded = (ArrayList<RecIntegral>) in.readObject();

        DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();

        for (RecIntegral rec : loaded) {
            arrInteg.add(rec);
            tModel.addRow(new Object[]{
                    rec.getLowLim(),
                    rec.getUpLim(),
                    rec.getStep(),
                    rec.geResult()
            });
        }

        JOptionPane.showMessageDialog(this, "Данные загружены (двоичный формат)");

    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Ошибка загрузки: " + ex.getMessage());
    }
}
}


/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
/* Create and display the form */
    java.awt.EventQueue.invokeLater(new Runnable() {
```

```
            public void run() {
                new NewJFrame().setVisible(true);
            }
        });
    }

    // Variables declaration - do not modify
    private javax.swing.JButton AddColection;
    private javax.swing.JButton DeleteAll;
    private javax.swing.JButton DownloadBinary;
    private javax.swing.JButton DownloadText;
    private javax.swing.JButton SaveBinary;
    private javax.swing.JButton SaveText;
    private javax.swing.JButton jButtonAdd;
    private javax.swing.JButton jButtonDelete;
    private javax.swing.JButton jButtonResult;
    private javax.swing.JFrame jFrame1;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JLabel jLabel3;
    private javax.swing.JLabel jLabel4;
    private javax.swing.JPanel jPanel1;
    private javax.swing.JScrollPane jScrollPane1;
    private javax.swing.JTable jTable1;
    private javax.swing.JTextField jTextFieldLow;
    private javax.swing.JTextField jTextFieldStep;
    private javax.swing.JTextField jTextFieldUp;
    // End of variables declaration
}
```

Класс RecIntegral:

```
package lab1.pkg1;
import java.io.Serializable;



public class RecIntegral  implements Serializable {


   private double lowLim;
   private double upLim;
   private double step;
   private double result;


   public RecIntegral(double lowLim, double upLim, double step) throws InvalidRec {
      if (!isValid(lowLim) || !isValid(upLim) || !isValid(step)) {
         String err ="";
         err = (!isValid(lowLim)) ? err + "Нижний предел: " + lowLim + "," : err;
         err = (!isValid(upLim)) ? err +  " Верхний предел: " +upLim + ",": err;
         err = (!isValid(step)) ? err +  " Шаг: " + step : err;
         throw new InvalidRec("Значения должны быть в диапазоне от 0.000001 до 1 000 000. Вы ввели: "
+ err);


      }
          if (upLim < lowLim)   throw new InvalidRec("Нижний предел больше верхнего");
      this.lowLim = lowLim;
      this.upLim = upLim;
      this.step = step;
      this.result = 0.0;


   }


   public RecIntegral(double lowLim, double upLim, double step, double result) throws InvalidRec {
      if (!isValid(lowLim) || !isValid(upLim) || !isValid(step)) {
```

```
        String err ="";
        err = (!isValid(lowLim)) ? err + "Нижний предел: " + lowLim + "," : err;
        err = (!isValid(upLim)) ? err +  " Верхний предел: " +upLim + ",": err;
        err = (!isValid(step)) ? err +  " Шаг: " + step : err;
        throw new InvalidRec("Значения должны быть в диапазоне от 0.000001 до 1 000 000. Вы ввели: "
+ err);
    }
    if (upLim < lowLim)   throw new InvalidRec("Нижний предел больше верхнего");
    this.lowLim = lowLim;
    this.upLim = upLim;
    this.step = step;
    this.result = result;
  }

  private boolean isValid(double value) {
    return value >= 0.000001 && value <= 1_000_000;
  }


  public double getLowLim() { return lowLim; }
  public double getUpLim() { return upLim; }
  public double getStep() { return step; }
  public double geResult() { return result; }


  public void setResult(double result) {
    this.result = result;
  }


  public double CalcIntegral(double lowLim, double upLim, double step) {
    double start = lowLim;
```

```
        double sumS = 0;

        do {
            double h = Math.min(step, (upLim - start));
            sumS += h * (1 / start + 1 / (start + h)) / 2;
            start += h;
        } while (start < upLim);

        return sumS;
    }
}
```

Класс NewThread

```
package lab1.pkg1;
import javax.swing.table.DefaultTableModel;
import javax.swing.JOptionPane;

public class NewThread extends Thread {

    private final RecIntegral rec;
    private final int rowIndex;
    private final DefaultTableModel model;

    public NewThread(RecIntegral rec, int rowIndex, DefaultTableModel model) {
        this.rec = rec;
        this.rowIndex = rowIndex;
        this.model = model;
    }

    @Override
    public void run() {
```

```
        JOptionPane.showMessageDialog(null, "Поток открылся");
        double result = rec.CalcIntegral(
                rec.getLowLim(),
                rec.getUpLim(),
                rec.getStep()
        );
        rec.setResult(result);

        javax.swing.SwingUtilities.invokeLater(() -> {
            model.setValueAt(result, rowIndex, 3);
        });
    }
}
```

РЕЗУЛЬТАТЫ ИСПЫТАНИЙ

Приложение Б

(обязательное)

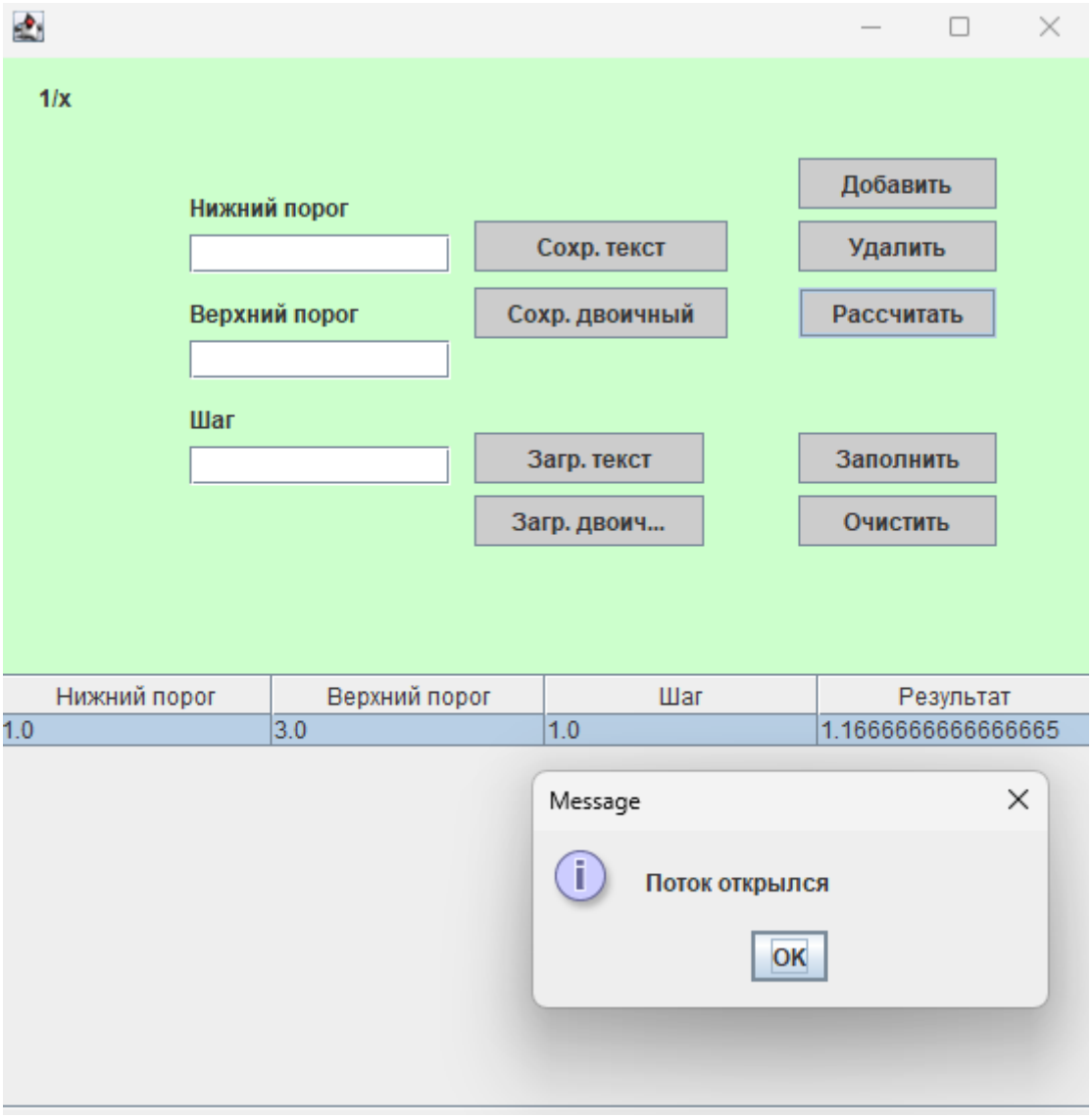

Рисунок Б.1 — Выполнение расчета в дополнительном потоке